

1. JWT (JSON Web Token) — Authentication ka backbone

JSON Web Token

Simple samajh:

JWT ek **digital identity card** hai jo server user ko deta hai login ke baad.

👉 Flow:

1. User login karta hai (email/password)
2. Server verify karta hai
3. Server ek **token generate karta hai**
4. Har request me user wo token bhejta hai

Node.js Implementation (Professional Flow)

Step 1: Token Generate karna

```
const jwt = require('jsonwebtoken');

const token = jwt.sign(
  { id: user._id, role: user.role }, // payload
  "SECRET_KEY",
  { expiresIn: "7d" }
);
```

Step 2: Middleware (Token Verify)

```
const verifyToken = (req, res, next) => {
  const token = req.headers.authorization;

  if (!token) return res.status(401).json({ message: "No token" });
```

```
try {
  const decoded = jwt.verify(token, "SECRET_KEY");
  req.user = decoded;
  next();
} catch (err) {
  res.status(401).json({ message: "Invalid token" });
}
};
```

Real Understanding:

- JWT = **stateless auth** (server session store nahi karta)
- Har request me token verify hota hai
- Mobile apps (Flutter) ke liye best

2. Password Hashing (bcrypt) — Security ka core

bcrypt

Simple samajh:

Password kabhi bhi **plain text me store nahi hota**
→ hum usko hash kar dete hain

Node.js Implementation

Step 1: Password Hash karna (Signup)

```
const bcrypt = require('bcrypt');

const hashedPassword = await bcrypt.hash(password, 10);
```

Step 2: Password Compare (Login)

```
const isMatch = await bcrypt.compare(password, user.password);

if (!isMatch) {
  return res.status(400).json({ message: "Invalid credentials" });
}
```

Real Understanding:

- Hash irreversible hota hai
- Even DB hack ho jaye → password safe
- Salt rounds (10) = security level

3. Role-Based Authorization (Admin/User)

Role-Based Access Control

Simple samajh:

Har user ko ek **role** milta hai:

- user
- admin

Aur APIs restrict hoti hain role ke hisaab se

Implementation (Middleware)

```
const authorizeRoles = (...roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ message: "Access denied" });
    }
  }
}
```

```
    next();  
  };  
};
```

Usage:

```
app.get('/admin', verifyToken, authorizeRoles("admin"), (req, res) =>  
{  
  res.send("Welcome Admin");  
});
```

Real Understanding:

- Auth ≠ Authorization
- JWT batata hai **tum kaun ho**
- Role batata hai **tum kya kar sakte ho**

MongoDB + Mongoose (Your Database Setup)

Tum use karoge:

- MongoDB
- Mongoose

User Model (Professional Level)

```
const mongoose = require('mongoose');  
  
const userSchema = new mongoose.Schema({  
  name: String,  
  email: {
```

```
    type: String,
    unique: true,
  },
  password: String,
  role: {
    type: String,
    enum: ["user", "admin"],
    default: "user",
  }
}, { timestamps: true });

module.exports = mongoose.model("User", userSchema);
```

Complete Auth Flow (Real World)

Signup:

1. User data aata hai
2. Password hash hota hai (bcrypt)
3. DB me save hota hai

Login:

1. Email find karo
2. Password compare karo
3. JWT generate karo
4. Token return karo

Protected Route:

1. Token verify (middleware)
2. Role check (optional)
3. API access



Professional Architecture Flow

Route → Controller → Service → Model (Mongoose)



JWT + bcrypt + Role check



Common Beginner Mistakes (Avoid These)

- ✗ Password plain text store karna
- ✗ JWT secret hardcode karna
- ✗ Token expire na karna
- ✗ Role check na lagana
- ✗ Middleware skip karna